

Begründung der Implementierung: Lagebild-Dashboard Katastrophenschutz Berlin

Autoren: Julian Schaefers, Til Scheerer · **Klasse:** LF8 - 11BE15 · **Version:** 2.0.0

Idee: Ein Dashboard, das die wichtigsten Lageinformationen für Berlin (Wetter, Pegel, Luft, Einsätze, amtliche Warnungen) auf einen Blick zusammenführt, statt mehrere Webseiten einzeln zu öffnen. Die Nutzer sind fiktiv, aber an realen Abläufen orientiert.

1. Anwender

	Primär: Leitstelle / Lagedienst	Sekundär: Einsatzkräfte im Außeneinsatz
Wer	Disponent:innen im Schichtdienst, Fachwissen hoch, Technikenntnis gering	Einsatz- und Zugführer:innen, viel im Freien
Gerät / Nutzung	Großer Monitor, dauerhaft im Hintergrund, Blick von 2–3 Sekunden	Smartphone/Tablet, kurz und gezielt unter Zeitdruck
Folge fürs Design	Alles ohne Scrollen sichtbar, ruhig, keine Bedienung nötig	Scrollbar, große Touch-Flächen, Details per Antippen

Nicht Zielgruppe sind Krisenstab (braucht Berichte) und Öffentlichkeit (dafür gibt es die NINA-App). Wegen des mehrsprachigen Einsatzumfelds und schlechter Lesbarkeit im Freien gibt es 5 Sprachen, hohen Kontrast, Hell-/Dunkelmodus und eine Sprachausgabe.

2. Inhalte, Datenquellen und Widgets

Nur, was die **erste Lagebeurteilung** unterstützt. Alle Quellen sind öffentlich und ohne API-Key nutzbar.

Inhalt / Widget	Warum relevant	Quelle
Wetter aktuell, 24-h-Verlauf, 7-Tage-Vorhersage	Sturm, Starkregen und Hitze bestimmen Einsatzlagen	Open-Meteo
Luftqualität (PM2.5/PM10)	Hinweis auf Rauchbelastung bei Bränden	Open-Meteo Air Quality
Pegel Spree (Köpenick), eingeordnet an amtlichen Kennwerten	Frühindikator für Hochwasser	PEGELONLINE (WSV)
Brände, Gesamteinsätze, Ø Reaktionszeit	Belastung der Feuerwehr; die Eintreffzeit ergänzt die Menge um einen Leistungsindikator	Berliner Feuerwehr Open Data
Amtliche Warnungen	Wichtigste Einzelinformation, vom Bund, mehrsprachig, mit Handlungsempfehlung	NINA/BBK
Gefahrenkarte Hochwasser	Räumlicher Bezug zum Pegel	Umweltatlas Berlin

Kennzahlen ohne verlässliche öffentliche Quelle haben wir bewusst weggelassen: Laufende Einsätze, Kräfte und Ressourcen sind internes Leitstellenwissen ohne Schnittstelle. Straßensperrungen (VIZ Berlin) wurden angebunden und getestet, aber verworfen, weil der Datensatz seit Juli 2025 nicht mehr aktualisiert wird – eine scheinbar aktuelle Kachel wäre irreführend.

3. Kriterien

Kriterium	Anforderung	Begründung
Übersichtlichkeit	Desktop (ab 1101 px): nichts scrollt, weder Seite noch Kachel noch Fenster – bei wenig Höhe wird es kompakter, mehrere Warnungen werden geblättert statt abgeschnitten	Vorgabe der Lernsituation, Blick unter Zeitdruck
Responsivität	Unter 1100 px normales Scrollen; Diagramme passen sich jeder Kachelgröße und jedem Zoom an	Außeneinsatz, unterschiedliche Monitore
Barrierefreiheit	5 Sprachen, hoher Kontrast, Sprachausgabe, Tastaturbedienung, Warnstufen nicht nur über Farbe	Mehrsprachige Teams, schlechtes Licht, Hände oft gebunden
Verständlichkeit	Kurzanleitung und Quellen per Info-Button, keine Anmeldung	Nutzbar ohne Schulung und Technikwissen
Zuverlässigkeit	Fällt eine Quelle aus, bleiben die anderen Kacheln nutzbar	Mehrere unabhängige externe Quellen
Transparenz	Quellen benannt, Stand jeder Kachel sichtbar, Hinweis „im Notfall 112“	Verfügbarkeit und Aktualität der externen Daten können wir nicht beeinflussen

4. Architektur: Frontend, Build-Prozess und (kein) Backend

Das Dashboard ist eine **statisch generierte Seite** mit **Astro und Node.js**. Es ist kein reines Frontend-Widget (z. B. iFrame), sondern ruft seine Daten selbst per API ab und wertet sie aus (z. B. Ø Reaktionszeit aus der Feuerwehr-CSV). Ein **Backend im eigentlichen Sinn gibt es nicht** – also keinen Dienst, der dauerhaft läuft und Anfragen zur Laufzeit beantwortet. Stattdessen gibt es drei Phasen:

Phase	Wo?	Aufgabe
Build-Zeit	GitHub Actions (CI/CD) bzw. lokal, in Node.js	Astro erzeugt fertiges HTML/CSS/JS, ruft dabei die NINA-Warnungen ab und bettet sie ein. Läuft einmal pro Build.
Auslieferung	GitHub Pages	Statischer Webserver, liefert nur die fertigen Dateien aus.
Laufzeit (Frontend)	Browser	Lädt Wetter, Luft, Pegel, Feuerwehrdaten direkt per API, zeichnet Diagramme, aktualisiert alle 5 Minuten.

Warum die Warnungen beim Build geladen werden: Browser setzen die *Same-Origin-Policy* durch: Eine Seite auf `github.io` darf die Antwort einer fremden Domain nur lesen, wenn deren Server das per CORS-Header (`Access-Control-Allow-Origin`) erlaubt. Open-Meteo, PEGELONLINE und GitHub senden diesen Header, `warnung.bund.de` nicht. Node.js kennt diese Browser-Regel nicht, deshalb funktioniert derselbe Abruf beim Build.

Folge für die Aktualität: Uhrzeit jede Sekunde; Wetter, Luft und Pegel alle 5 Minuten; Feuerwehrdaten alle 5 Minuten abgefragt, die Quelle liefert aber nur **Vortageswerte**. Die Warnungen sind nur so aktuell wie der letzte Build: Der Zeitplan fordert alle 30 Minuten einen Build an, GitHub führt geplante Läufe aber nicht garantiert pünktlich aus – **gemessen lagen 3 bis 7 Stunden dazwischen**. Das ist die schwächste Stelle, deshalb machen wir sie sichtbar: Die Warnungen-Kachel zeigt das Alter („vor 4 Std.“), ab einer Stunde erscheint ein Hinweis in Warnfarbe mit Verweis auf `warnung.bund.de` und die NINA-App.

Begründung: GitHub Pages ist kostenlos, wartungsarm und braucht keinen eigenen Server. Für einen Prototyp wiegt das schwerer als Echtzeit-Warnungen; den Nachteil haben wir bewusst in Kauf genommen und offen benannt. Alle anderen Kennzahlen sind davon nicht betroffen.

5. Weitere Implementierungsentscheidungen

- **Getrennte Datenbereiche:** Jede Quelle wird einzeln geladen und fehlerbehandelt; fällt eine aus, zeigt nur diese Kachel „Keine Daten“.
- **Diagramme ohne Bibliothek:** Eigene SVG-Diagramme, die bei jeder Größenänderung ihrer Kachel neu zeichnen und so bei jedem Zoom unverzerrt bleiben.
- **Pegellatte statt Verlauf:** Die Spree ist in Köpenick reguliert (30 Tage: 85–89 cm), ein Verlauf zeigte fast nur Messrauschen. Die Kachel ordnet den Stand an den amtlichen Kennwerten ein (MNW, MW, MHW, HHW) und zeigt ihn als gefüllten Flussquerschnitt. Die Bereiche normal/erhöht/hoch sind eine eigene Einteilung, da PEGELONLINE für Köpenick keine Meldestufen liefert.
- **Sprachausgabe in zwei Stufen:** Zuerst die gezielt gewählte Stimme des Browsers (sonst liest z. B. Chrome französischen Text englisch vor). Fehlt sie (z. B. Firefox ohne Sprachpaket), lässt sich per Knopf ein lokales Sprachmodell (Piper TTS) laden, das danach im Browser gespeichert bleibt. **Nur auf Klick:** Die erste Stimme kostet rund 71 MB (LTE 1–2 Minuten, 3G mit 1,6 Mbit/s 9 Minuten) – auf Mobilfunk zu viel, um es unbemerkt auszulösen. Geladen wird in 2-MB-Abschnitten mit Wiederholung, damit auch langsame Verbindungen ans Ziel kommen.
- **Mehrsprachigkeit und Barrierefreiheit:** Alle Sprachfassungen liegen nach dem Laden vor; Kontrast und Farbschema schalten nur CSS-Variablen um; alle Fenster sind per Tastatur bedienbar.
- **Konfiguration:** Lokal ist keine nötig; optionale Einstellungen stehen in `.env` (Vorlage `.env.example`), Hosting-Pfade in `astro.config.mjs`.

6. Testen, Dokumentation und Anwenderschulung

Testen: Manuell im Browser und über den Produktions-Build; automatisierte Tests gibt es noch nicht. Werte müssen mit der Originalquelle übereinstimmen, bei Ausfall einer Schnittstelle müssen die übrigen Kacheln weiterlaufen; für Warnungen gibt es einen Testmodus. Geprüft wurden Desktop bis Smartphone, Browser-Zoom 100–200 %, alle Sprachen, Farbmodi und Tastaturbedienung. Ladezeiten haben wir mit gedrosselter Verbindung von 3G bis Glasfaser gemessen (Seitenaufruf 0,3 s ungedrosselt bis 14 s bei langsamem 3G).

Dokumentation: README für Administration/Entwicklung (Installation, Konfiguration, Datenquellen, Architektur, Deployment), Kurzanleitung im Dashboard für Anwender, beides mit offen genannten Grenzen der Daten.

Anwenderschulung: Kurz und praxisnah: was sich automatisch aktualisiert, was anklickbar ist, wie Sprache, Kontrast und Tastatur funktionieren. Wichtig sind die Grenzen: Feuerwehrdaten sind Vortageswerte, Warnungen können mehrere Stunden alt sein, und das Dashboard ersetzt weder NINA-App noch Notruf 112.

7. Datenschutz, Wartung und geplante Erweiterungen

Datenschutz: Keine Nutzerdaten, keine Anmeldung; der Browser merkt sich nur Sprache, Farbschema und Kontrast. Ein Sprachmodell liegt im Browser-Speicher, der Text wird lokal berechnet und an keinen Dienst geschickt. Alle Quellen sind öffentlich und ohne API-Schlüssel.

Wartung: Externe Schnittstellen können sich ändern oder ausfallen (siehe Straßensperrungen) und müssen regelmäßig geprüft werden. GitHub deaktiviert geplante Workflows nach 60 Tagen ohne Aktivität; die Altersanzeige der Warnungen macht das sofort sichtbar.

Geplante Erweiterungen: Vor allem **Warnungen in Echtzeit** über ein kleines echtes Backend: eine Serverless-Funktion (z. B. Cloudflare Worker), die die Anfrage an `warnung.bund.de` weiterleitet und den fehlenden CORS-Header ergänzt. Dann lädt der Browser die Warnungen alle 5 Minuten wie alle anderen Daten. Nicht umgesetzt, weil dafür ein zusätzlicher Dienst betrieben und überwacht werden muss. Außerdem denkbar: Benachrichtigungen bei neuen Warnungen oder hohem Pegel, weitere Quellen, automatisierte Tests und mit Anmeldung auch interne Leitstellendaten.